Case Study 1: Student Email and Identifier Validation

1. Problem Statement

To write a Python program to check whether a student's email address and registration number are valid or invalid. The program checks for missing parts, wrong symbols, and incorrect characters.

2. Objectives

1. To check the student's email address.
2. To check the student's registration number.
3. To find missing parts and wrong symbols.
4. To display whether the details are valid or invalid.

3. Assumptions

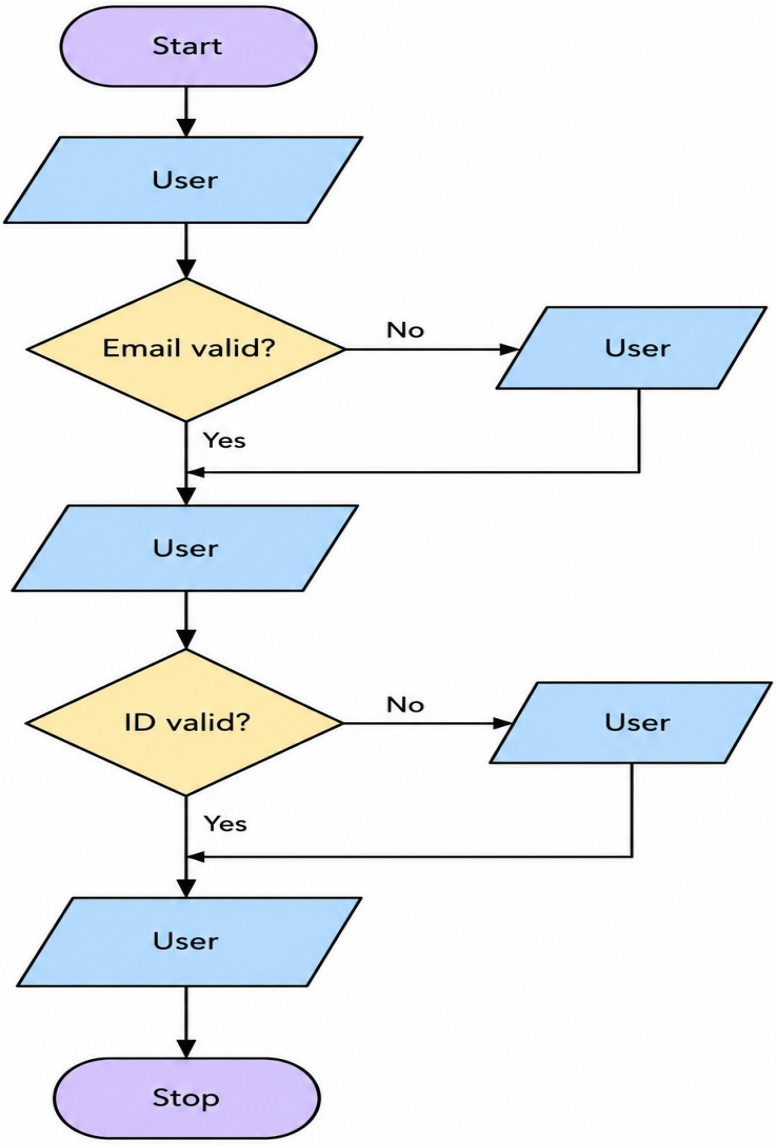
1. The student enters an email address and registration number.
2. The email must contain @ and a dot (.).
3. The email must not contain spaces.
4. The registration number must contain letters and numbers in the correct format.
5. Empty inputs are considered invalid.

4. Algorithm

1. Start.
2. Enter the student email address.
3. Enter the registration number.
4. Check whether the email contains @ and a dot.
5. Check whether the email contains spaces or missing parts.
6. Check whether the registration number follows the correct format.
7. Display whether the email is valid or invalid.

8. Display whether the registration number is valid or invalid.
9. Stop.

5.Flowchart



6.Pseudocode

START

INPUT email

INPUT registration number

IF email is valid THEN

DISPLAY "Email is valid"

ELSE

DISPLAY "Email is invalid"

END IF

IF registration number is valid THEN

DISPLAY "Registration number is valid"

ELSE

DISPLAY "Registration number is invalid"

END IF

STOP

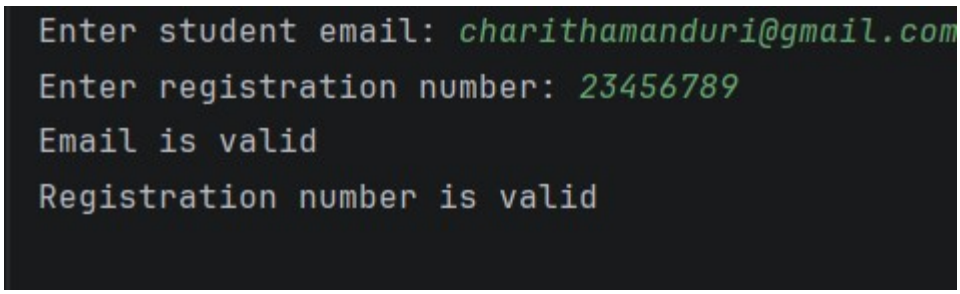
7.Python source code

```
email = input("Enter student email: ")
reg = input("Enter registration number: ")
if "@" in email and "." in email and " " not in email:
    print("Email is valid")
else:
    print("Email is invalid")
if reg.isalnum() and len(reg) > 0:
    print("Registration number is valid")
else:
    print("Registration number is invalid")
```

8.Test Cases

Test case	Email input	Registration number	Expected output
1	`charitha@gmail.com`	`CSM2026001`	Both valid
2	`charithagmail.com`	`CSM2026001`	Email invalid, registration valid
3	`charitha@gmailcom`	`CSM2026001`	Email valid according to basic code, registration valid

9.Sample Output



10.Conclusion

In this program, I learned how to check student email addresses and registration numbers using Python. I used simple conditions like if-else, the and operator, and the isalnum() method to identify valid and invalid inputs. By testing different inputs, I understood how to find some common mistakes in student details. This program helped me understand the basic concept of input validation in Python.

Case Study 2: Student Feedback Sentiment Scoring

1. Problem Statement

To write a Python program to analyse student feedback sentences using positive and negative words. The program counts positive and negative words, calculates the feedback score, and classifies the feedback as positive, negative, or neutral.

2. Objectives

1. To take student feedback as input.
2. To identify positive and negative words in the feedback.
3. To count the positive and negative words.
4. To calculate the feedback score.
5. To classify the feedback as positive, negative, or neutral.

3. Assumptions

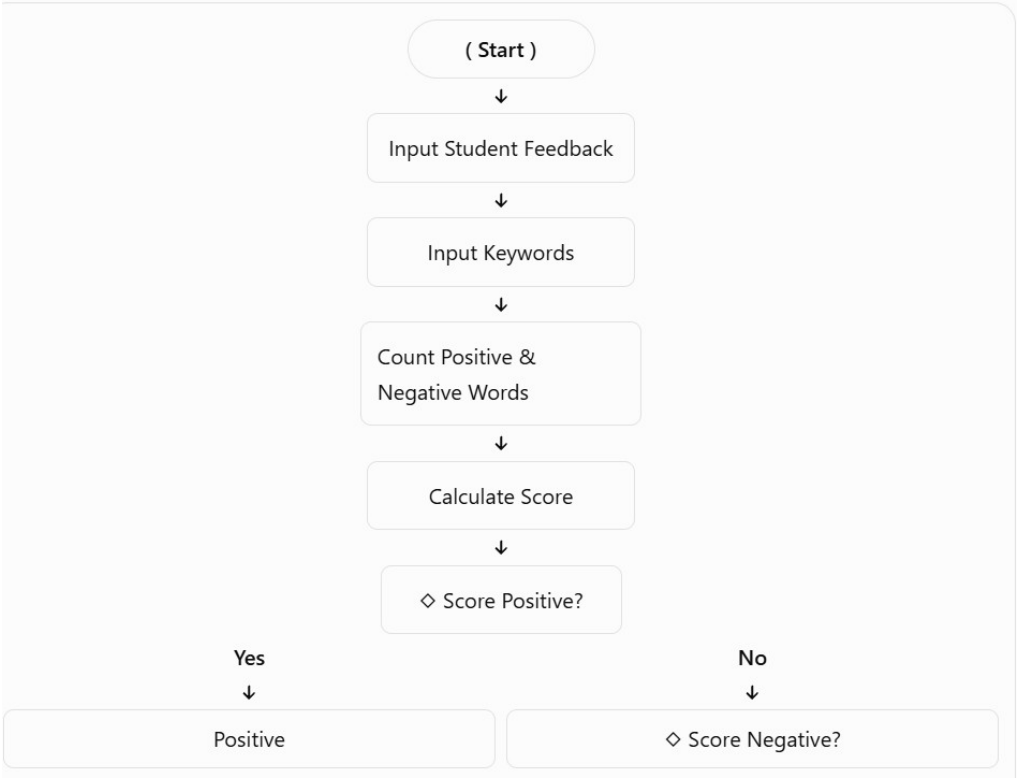
1. The student enters feedback in simple English.
2. A list of positive and negative words is already given.
3. Positive words increase the feedback score.
4. Negative words decrease the feedback score.
5. If the score is greater than zero, the feedback is positive.
6. If the score is less than zero, the feedback is negative.
7. If the score is zero, the feedback is neutral.
8. The program checks the words in the feedback and does not understand the full meaning of the sentence.

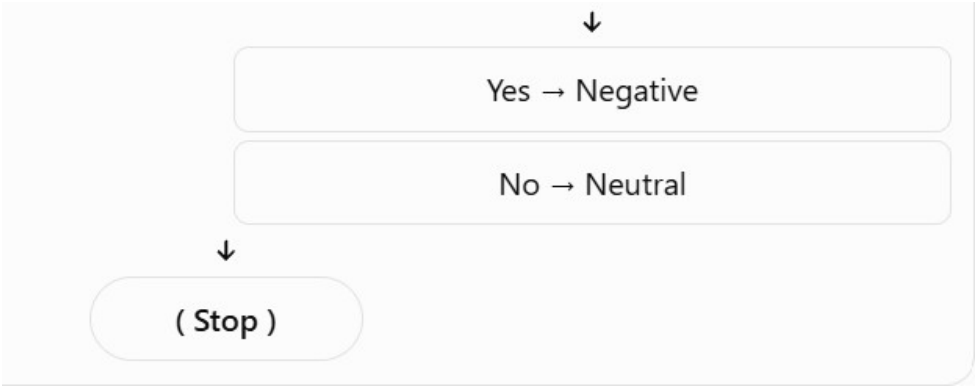
4. Algorithm

1. Start.
2. Create a list of positive words and negative words.

3. Enter the student feedback.
4. Set the positive count and negative count to zero.
5. Check each word in the feedback.
6. If a word is in the positive list, increase the positive count.
7. If a word is in the negative list, increase the negative count.
8. Calculate the score by subtracting the negative count from the positive count.
9. If the score is greater than zero, display Positive Feedback.
10. If the score is less than zero, display Negative Feedback.
11. Otherwise, display Neutral Feedback.
12. Display the positive count, negative count, and score.
13. Stop.

5.Flowchart





6.Pseudocode

START

INPUT feedback

COUNT positive words

COUNT negative words

score = positive - negative

IF score > 0

  DISPLAY "Positive Feedback"

ELSE IF score < 0

  DISPLAY "Negative Feedback"

ELSE

  DISPLAY "Neutral Feedback"

END IF

STOP

7.Python Source Code

```
feedback = input("Enter feedback: ")
positive = 0
negative = 0
for word in feedback.split():
    if word in ["good", "happy", "excellent", "nice"]:
        positive = positive + 1
    elif word in ["bad", "poor", "sad", "worst"]:
        negative = negative + 1
score = positive - negative
print("Positive words:", positive)
print("Negative words:", negative)
print("Score:", score)
if score > 0:
    print("Positive Feedback")
elif score < 0:
    print("Negative Feedback")
else:
    print("Neutral Feedback")
```

8.Test cases

S.No.	Input Feedback	Expected Output
1	good teaching and nice classes	Positive Feedback
2	bad teaching and poor classes	Negative Feedback
3	good and bad	Neutral Feedback
4	excellent teaching	Positive Feedback

9.Sample output

```
Enter feedback: good
Positive words: 1
Negative words: 0
Score: 1
Positive Feedback
```

10.Conclusion:

In this program, I learned how to check student feedback using Python. I counted positive and negative words and calculated the score to display whether the feedback is positive, negative, or neutral.

Case study 2: **Hospital Ward Occupancy Monitoring**

1. Problem Statement

A hospital has different wards with a limited number of beds. This program checks how many beds are occupied and displays the bed usage and capacity status.

2. Objectives

- To store occupied bed details.
- To calculate bed usage percentage.
- To check whether beds exceed the capacity.
- To identify when all beds are occupied.

3. Assumptions

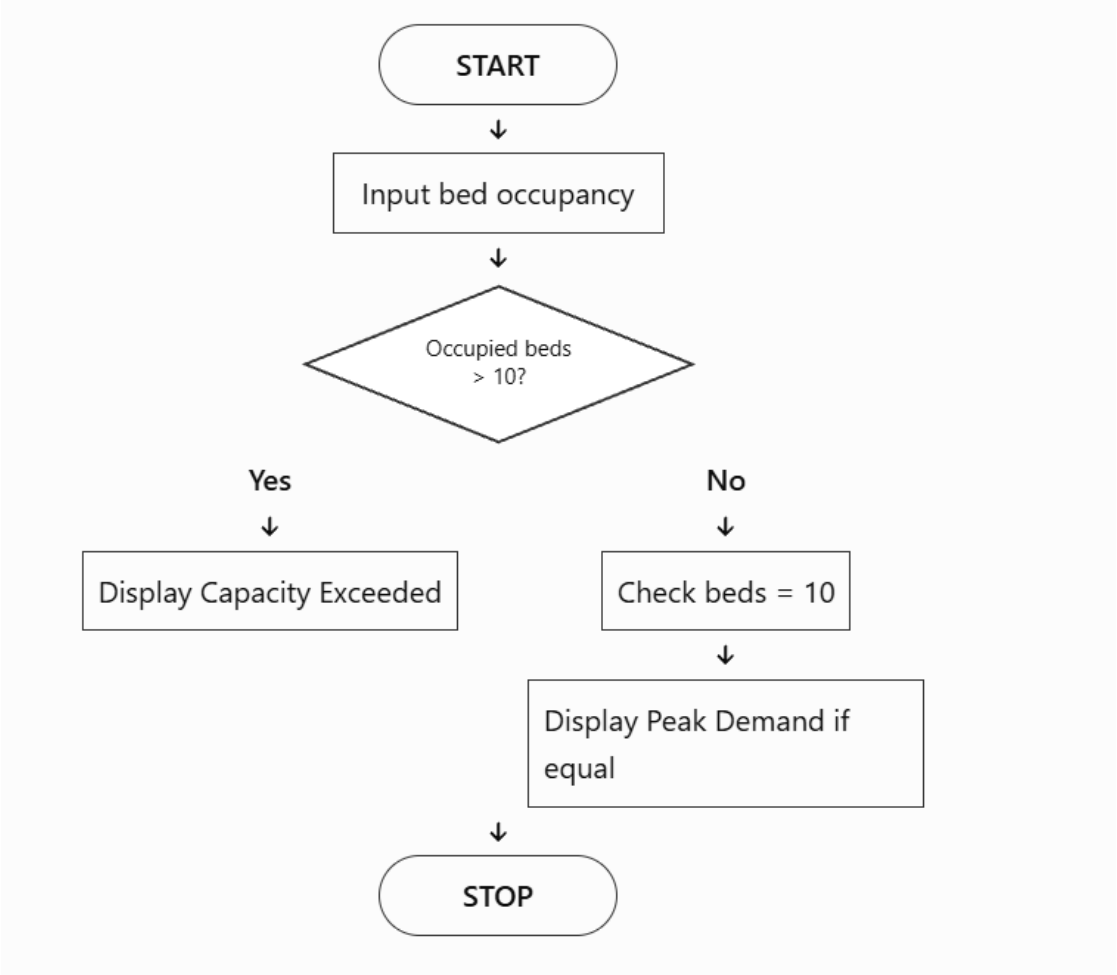
- Each ward has 10 beds.
- Bed details are stored in a list.
- Bed usage percentage is calculated using occupied beds and total beds.
- If occupied beds are greater than 10, capacity is exceeded.
- If occupied beds are equal to 10, it is peak demand.

4. Algorithm

1. Start.
2. Store occupied bed values in a list.
3. Set capacity to 10.
4. Read the occupied bed values.
5. Calculate the bed usage percentage.
6. If occupied beds are greater than capacity, display "Capacity exceeded".
7. If occupied beds are equal to capacity, display "Peak demand".

8. Stop.

5.Flowchart



6.Pseudocode

START

Store bed values in a list

Set capacity = 10

FOR each bed value

Calculate usage percentage

Display occupied beds and percentage

IF occupied beds > 10

Display "Capacity exceeded"

END IF

IF occupied beds = 10

Display "Peak demand"

END IF

END FOR

STOP

7.Python source code

```
beds = [[8, 9, 10], [6, 7, 8]]
capacity = 10
for ward in beds:
    for occupied in ward:
        percentage = (occupied / capacity) * 100
        print("Occupied beds:", occupied)
        print("Utilization:", percentage)
        if occupied > capacity:
            print("Capacity exceeded")
        if occupied == capacity:
            print("Peak demand")
```

8.Test cases

S.No.	Occupied Beds	Expected Output
1	8	Utilization: 80.0
2	9	Utilization: 90.0
3	10	Peak demand

9.Sample output

```
Occupied beds: 8
Utilization: 80.0
Occupied beds: 9
Utilization: 90.0
Occupied beds: 10
Utilization: 100.0
Peak demand
Occupied beds: 6
Utilization: 60.0
Occupied beds: 7
```

```
Peak demand
Occupied beds: 6
Utilization: 60.0
Occupied beds: 7
Utilization: 70.0
Occupied beds: 8
Utilization: 80.0
```

10.Conclusion

In this program, I learned how to calculate bed usage percentage and check hospital ward occupancy using Python. I also learned how to identify peak demand and capacity violations.

Case Study: Food Waste Monitoring System

1. Problem Statement

This program checks prepared meals, served meals, and leftover food in a college canteen. It calculates food waste and identifies days with high waste.

2. Objectives

- To store prepared and served meals.
- To calculate leftover food.
- To calculate waste percentage.
- To identify high food waste.
- To display the result.

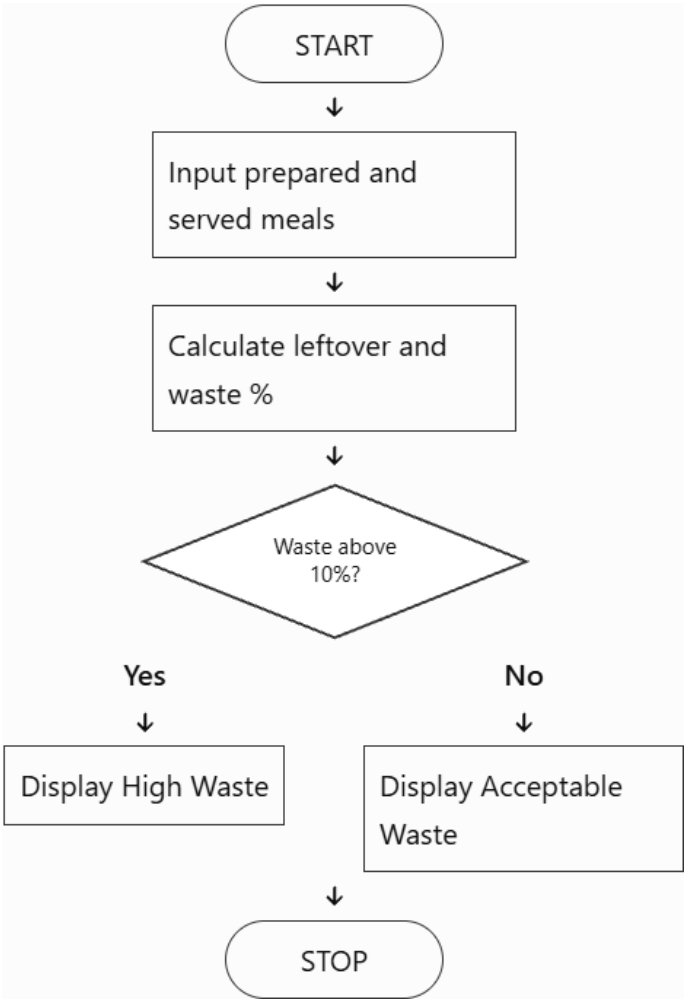
3. Assumptions

- Meal details are stored in lists.
- Leftover = Prepared meals − Served meals.
- Waste percentage = (Leftover / Prepared meals) × 100.
- The waste limit is 10%.
- If waste is more than 10%, it is High Waste.

4. Algorithm

1. Start.
2. Store prepared and served meals.
3. Calculate leftover meals.
4. Calculate waste percentage.
5. Check whether waste is more than 10%.
6. Display High Waste or Acceptable Waste.
7. Repeat for all days.
8. Stop.

5. Flowchart



6.Pseudocode

START

Store prepared and served meals

FOR each day

Calculate leftover

Calculate waste percentage

IF waste > 10

Display "High Waste"

ELSE

Display "Acceptable Waste"

END IF

END FOR

STOP

7.Python source code

```
prepared = [100, 120, 90, 110]
served = [95, 100, 80, 105]
for i in range(4):
    leftover = prepared[i] - served[i]
    waste = (leftover / prepared[i]) * 100
    print("Day:", i + 1)
    print("Leftover:", leftover)
    print("Waste:", round(waste, 2))
    if waste > 10:
        print("High Waste")
    else:
        print("Acceptable Waste")
```

8.Test cases

Day	Prepared	Served	Leftover	Expected Result
1	100	95	5	Acceptable Waste
2	120	100	20	High Waste
3	90	80	10	High Waste
4	110	105	5	Acceptable Waste

9.Sample output



10.Conclusion

In this program, I learned how to calculate leftover food and waste percentage using Python. I also learned how to identify high food waste using an if-else condition

Study case 5:Sports Tournament Ranking System

1. Problem Statement

This program calculates team points and ranks teams based on their performance.

2. Objectives

- Store team details.
- Calculate team points.
- Calculate goal difference.
- Display final ranking.

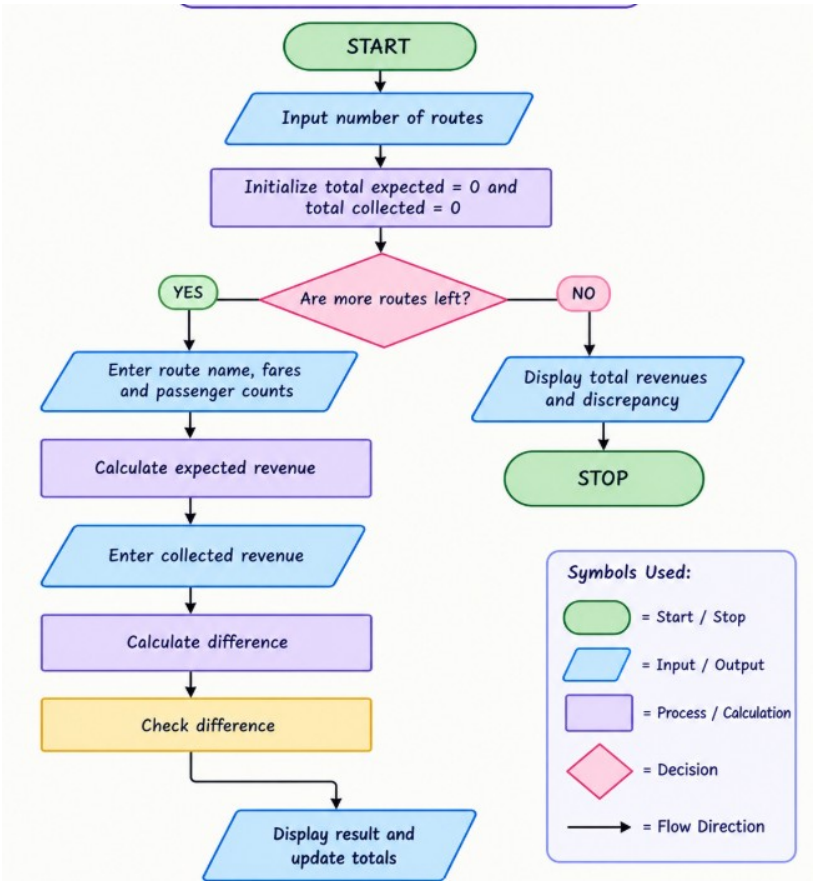
3. Assumptions

- Win = 2 points
- Draw = 1 point
- Loss = 0 points
- Goal Difference = Goals Scored − Goals Conceded

4. Algorithm

1. Start.
2. Enter number of teams.
3. Enter team details.
4. Calculate total points.
5. Calculate goal difference.
6. Arrange teams by points.
7. If points are equal, compare goal difference.
8. Display final ranking.
9. Stop.

5.Flowchart



6.Pseudocode

START

INPUT number of teams

INPUT team details

CALCULATE points

CALCULATE goal difference

ARRANGE teams by points

IF points are equal

CHECK goal difference

END IF

DISPLAY final ranking

STOP

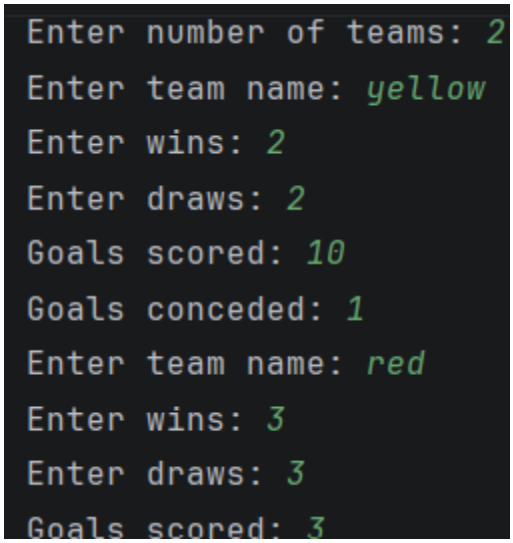
7.Python source code

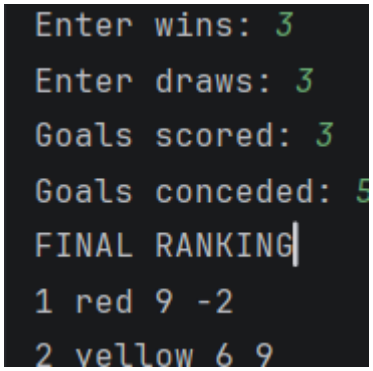
```
teams = []
n = int(input("Enter number of teams: "))
for i in range(n):
    name = input("Enter team name: ")
    wins = int(input("Enter wins: "))
    draws = int(input("Enter draws: "))
    scored = int(input("Goals scored: "))
    conceded = int(input("Goals conceded: "))
    points = wins * 2 + draws
    diff = scored - conceded
    teams.append([name, points, diff])
teams.sort(key=lambda x :{__getitem__} : (x[1], x[2]), reverse=True)
print("FINAL RANKING")
for i, team in enumerate(teams, 1):
    print(i, team[0], team[1], team[2])
```

8.Test cases

Team	Wins	Draws	Goals Scored	Goals Conceded
Lions	4	1	10	5
Tigers	4	1	8	5
Eagles	2	2	6	4
Sharks	1	1	4	7

9.Sample output





10.conclusion

This program calculates team points, compares goal differences, and displays the final tournament ranking using basic Python concepts.